

Universidad del Valle de Guatemala

CC2008 - Introducción a la Programación Orientada a Objetos

Semestre II, 2026

AGENCIA DE DETECTIVES – CASO MISTERIOSO

Laboratorio 2 - Arreglos y excepciones

Josue David Garcia Mendez 26375

El programa organizará un caso de una agencia de detectives mediante cuatro clases: Principal, Caso, Ubicacion y Pista. Este documento define los requisitos, responde las diez preguntas de análisis y presenta el diseño UML que guiará la implementación en Java.

Documento de trabajo. La guía exige entregar el análisis y diseño en PDF usando la plantilla del curso. Esa plantilla no forma parte del material adjunto; este contenido deberá adaptarse a ella antes de la entrega. El código Java se desarrollará en una etapa posterior.

Requisitos funcionales

Los siguientes 24 puntos recogen los requisitos de la guía e incorporan las condiciones del menú y del apartado Programa.

1. Solicitar al inicio el nombre del caso, su código y el nombre del detective responsable.
2. Administrar un máximo de cinco ubicaciones mediante un arreglo básico de objetos.
3. Guardar en cada ubicación su código, nombre, dirección o descripción, nivel de riesgo y estado.
4. Exigir un riesgo entre 1 y 10 inclusive. Un valor fuera de este rango debe producir `IllegalArgumentException` al crear o modificar la ubicación.
5. Registrar una ubicación en la posición elegida por el usuario, verificando los límites del arreglo, la disponibilidad del espacio y la validez del objeto.
6. Mantener null en cada posición vacía y comprobarlo antes de utilizar una ubicación.
7. Consultar todas las ubicaciones registradas e indicar sus posiciones, omitiendo los espacios que contengan null.
8. Consultar una ubicación por su posición e informar si el índice es inválido o el espacio está vacío, sin terminar inesperadamente.
9. Modificar el riesgo y el estado de una ubicación existente, respetando las mismas validaciones.
10. Descartar una ubicación y dejar null en la posición que ocupaba, para permitir su reutilización.
11. Administrar una cantidad variable de pistas mediante `ArrayList<Pista>`, utilizando `ArrayList` directamente y sin declarar la colección como `List`.
12. Guardar en cada pista su código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
13. Validar importancia entre 1 y 10 y confiabilidad entre 0 y 100, ambos inclusive. Los niveles inválidos deben producir `IllegalArgumentException`.
14. Registrar pistas nuevas sin permitir códigos repetidos dentro del mismo caso.
15. Consultar todas las pistas e informar de forma clara cuando todavía no exista ninguna.
16. Buscar una pista por código recorriendo el `ArrayList`.
17. Modificar la información de una pista existente y aplicar las validaciones utilizadas al crearla.

18. Localizar por código y eliminar del ArrayList una pista descartada o falsa.
19. Mostrar un reporte con siete resultados: ubicaciones registradas, espacios disponibles, ubicación de mayor riesgo, pistas registradas, pista de mayor importancia, pista de mayor confiabilidad y promedio de importancia.
20. Calcular los resultados de pistas usando solo los elementos del ArrayList. Si está vacío, evitar máximos y promedio que requieran elementos.
21. Leer números mediante Scanner y manejar InputMismatchException; limpiar la entrada incorrecta y permitir reintentar.
22. Usar try-catch para manejar las entradas incorrectas y las excepciones de validación de los objetos.
23. Incluir al menos un finally para ejecutar una acción final pertinente tanto si ocurre un error como si la operación termina correctamente.
24. Continuar después de errores manejados y ofrecer las trece opciones del menú. Nuevo caso debe reemplazar el caso actual por otro con ambas colecciones vacías; Salir debe finalizar la ejecución.

Orden del menú: 1 Nuevo caso; 2 Registrar ubicación; 3 Consultar ubicaciones; 4 Consultar una ubicación; 5 Modificar ubicación; 6 Descartar ubicación; 7 Registrar pista; 8 Consultar pistas; 9 Buscar pista; 10 Modificar pista; 11 Eliminar pista; 12 Mostrar reporte de investigación; 13 Salir.

1.Propiedades y métodos de cada clase

Las responsabilidades se separan para que la consola se encargue de la interacción y las clases del modelo administren los datos y sus reglas. No se necesita una clase Detective: la guía solo exige guardar el nombre del responsable.

Clase	Propósito
Principal	Es el driver del programa. Lee la entrada, presenta el menú y comunica los resultados o errores al usuario.
Caso	Almacena los datos del caso, administra ambas colecciones y calcula los resultados de la investigación.
Ubicacion	Representa un lugar de investigación y garantiza que su nivel de riesgo sea válido.
Pista	Representa una evidencia y valida sus niveles de importancia y confiabilidad.

Propiedades de Caso

Atributo	Tipo	Propósito
MAX_UBICACIONES	int	Constante de clase con valor 5 que fija la capacidad del arreglo.
nombre	String	Nombre que identifica descriptivamente la investigación.
codigo	String	Código de identificación del caso.
detectiveResponsable	String	Nombre del detective a cargo del caso.

ubicaciones	Ubicacion[]	Arreglo de cinco posiciones; cada una contiene una referencia a Ubicacion o null.
pistas	ArrayList<Pista>	Colección dinámica de pistas cuyos códigos no pueden repetirse.

Propiedades de Principal

Atributo	Tipo	Propósito
scanner	Scanner	Único lector de System.in, compartido por las operaciones de la consola.
casoActual	Caso	Referencia al caso activo. Se reemplaza únicamente después de construir un nuevo caso válido.

Todos los atributos serán private. MAX_UBICACIONES será static final. Los atributos marcados como invariables en el diseño serán final; en las colecciones, final fija la referencia, pero permite cambiar su contenido mediante operaciones de Caso.

Los métodos se detallan en la respuesta 2 junto con su visibilidad, parámetros, retorno y propósito. Esa especificación completa la identificación solicitada en esta pregunta.

Propiedades de las ubicaciones y las pistas

Propiedades de ubicación

Atributo	Tipo	Propósito
codigo	String	Identifica la ubicación. Se conserva durante su existencia.
nombre	String	Nombre del lugar que se mostrará al consultar la ubicación.
direccion	String	Dirección o descripción que permite reconocer el lugar.
nivelRiesgo	int	Grado de riesgo del lugar, entre 1 y 10 inclusive.
estado	String	Situación de la investigación del lugar, escrita por el usuario.

Propiedades de Pista

Atributo	Tipo	Propósito
codigo	String	Identificador estable para buscar, modificar o eliminar la pista.
descripcion	String	Explica qué se encontró durante la investigación.
tipoEvidencia	String	Clasificación escrita por el usuario, por ejemplo documental o física.
nivelImportancia	int	Relevancia para el caso, entre 1 y 10 inclusive.
nivelConfiabilidad	int	Confiabilidad expresada como porcentaje entero de 0 a 100 inclusive.

Decisiones de diseño

Los textos se recortarán con trim y no podrán ser null ni estar vacíos. Estado y tipo de evidencia serán textos libres porque la guía no fija un catálogo de valores. Los tres niveles serán enteros; el promedio será decimal.

Los códigos de pista permanecerán fijos después de crear el objeto. Modificar una pista permitirá cambiar descripción, tipo, importancia y confiabilidad. Los códigos se compararán sin distinguir mayúsculas y minúsculas y sin espacios en los extremos; por ejemplo, P01 y p01 identificarán la misma pista.

Estas son decisiones de la propuesta, no requisitos adicionales expresos del enunciado. La guía exige unicidad de códigos de pistas; no establece esa condición para los códigos de ubicaciones.

2 Tipos y métodos de ubicación y pista

La notación de las firmas es nombre(parámetro: Tipo): Retorno. El signo + indica public y el signo - indica private. Los constructores no tienen tipo de retorno; void significa que un método no devuelve un valor.

Metodos de ubicación

Firma y visibilidad	Propósito y resultado
+ Ubicacion(codigo: String, nombre: String, direccion: String, nivelRiesgo: int, estado: String)	Inicializa los cinco atributos después de validarlos. Lanza IllegalArgumentException si algún dato es inválido.
+ getCodigo(): String	Devuelve el código de la ubicación.
+ getNombre(): String	Devuelve el nombre del lugar.
+ getDireccion(): String	Devuelve su dirección o descripción.
+ getNivelRiesgo(): int	Devuelve el riesgo para consultas y comparaciones del reporte.
+ getEstado(): String	Devuelve el estado actual del lugar.
+ actualizar(nivelRiesgo: int, estado: String): void	Valida ambos valores antes de modificar el riesgo y el estado. Lanza IllegalArgumentException si alguno no es válido.
+ toString(): String	Devuelve un resumen con los cinco atributos para mostrarlo en consola.

Metodos de pista

Firma y visibilidad	Propósito y resultado
+ Pista(codigo: String, descripcion: String, tipoEvidencia: String, nivelImportancia: int, nivelConfiabilidad: int)	Valida e inicializa la evidencia. Lanza IllegalArgumentException por textos inválidos o niveles fuera de rango.
+ getCodigo(): String	Devuelve el identificador utilizado en las búsquedas.
+ getDescripcion(): String	Devuelve la descripción de la evidencia.
+ getTipoEvidencia(): String	Devuelve su clasificación.
+ getNivelImportancia(): int	Devuelve la importancia para el máximo y el promedio.
+ getNivelConfiabilidad(): int	Devuelve la confiabilidad para consultas y comparaciones.
+ actualizar(descripcion: String, tipoEvidencia: String, nivelImportancia: int, nivelConfiabilidad: int): void	Valida los cuatro datos antes de asignarlos. Conserva el código y lanza IllegalArgumentException si hay información inválida.

+ toString(): String

Devuelve un resumen con los cinco atributos para mostrarlo en consola.

Tipos y métodos de caso para ubicaciones

Caso administra la estructura y aplica las reglas de posición y disponibilidad. Ubicacion se encarga de validar sus propios datos.

Firma y visibilidad	Propósito y resultado
+ Caso(nombre: String, codigo: String, detectiveResponsable: String)	Valida los tres textos y crea un arreglo de cinco posiciones null y un ArrayList vacío. Lanza IllegalArgumentException por textos inválidos.
+ getUbicaciones(): Ubicacion[]	Devuelve una copia del arreglo conservando sus cinco posiciones, para consultar sin exponer el contenedor interno.
+ registrarUbicacion(posicion: int, ubicacion: Ubicacion): void	Comprueba el índice, que el espacio esté libre y que el objeto no sea null. Lo almacena; lanza IllegalArgumentException por datos inválidos e IllegalStateException si está ocupado.
+ consultarUbicacion(posicion: int): Ubicacion	Devuelve el objeto de un índice válido, o null si está vacío. Lanza IllegalArgumentException si el índice no existe.
+ modificarUbicacion(posicion: int, nivelRiesgo: int, estado: String): void	Comprueba la posición y llama a actualizar. Lanza IllegalStateException si no hay ubicación e IllegalArgumentException por índice o datos inválidos.
+ descartarUbicacion(posicion: int): boolean	Asigna null y devuelve true si había un objeto; devuelve false si ya estaba vacía. Lanza IllegalArgumentException por índice inválido.
+ contarUbicaciones(): int	Recorre el arreglo y cuenta únicamente las referencias distintas de null.
+ contarEspaciosDisponibles(): int	Devuelve MAX_UBICACIONES menos contarUbicaciones().
+ obtenerUbicacionMayorRiesgo(): Ubicacion	Devuelve la ubicación de mayor riesgo; omite los null. Devuelve null si no existe ninguna.
+ toString(): String	Devuelve nombre, código y detective responsable para identificar el caso activo.
- validarPosicion(posicion: int): void	Exige $0 \leq \text{posicion} < \text{ubicaciones.length}$ antes de acceder al arreglo. Lanza IllegalArgumentException si no se cumple.

Las posiciones visibles serán de 0 a 4, igual que los índices de Java. El menú lo indicará expresamente. Descartar una ubicación no desplazará las demás: únicamente cambiará a null la posición seleccionada.

Tipo y métodos de caso para pistas

Firma y visibilidad	Propósito y resultado
+ getPistas(): ArrayList<Pista>	Devuelve una copia del ArrayList para consultar sus elementos sin exponer el contenedor interno.
+ registrarPista(pista: Pista): void	Rechaza null y códigos ya existentes con IllegalArgumentException. Si la pista es válida, la agrega con add.
+ buscarPista(codigo: String): Pista	Usa la búsqueda por índice y devuelve la pista encontrada, o null si no existe. Rechaza un código vacío o null.

+ modificarPista(codigo: String, descripcion: String, tipoEvidencia: String, nivelImportancia: int, nivelConfiabilidad: int): boolean	Busca por código y actualiza los otros cuatro atributos. Devuelve true si se modificó y false si no existe; los datos inválidos generan IllegalArgumentException.
+ eliminarPista(codigo: String): boolean	Busca el índice y elimina con remove(indice). Devuelve true si la eliminó y false si no existe. Rechaza un código vacío o null.
+ contarPistas(): int	Devuelve pistas.size(), que representa la cantidad real de elementos registrados.
+ obtenerPistaMayorImportancia(): Pista	Recorre las pistas y devuelve la de mayor importancia, o null si la colección está vacía.
+ obtenerPistaMayorConfiabilidad(): Pista	Recorre las pistas y devuelve la de mayor confiabilidad, o null si la colección está vacía.
+ calcularPromedioImportancia(): double	Suma la importancia con un acumulador double y divide entre pistas.size(). Lanza IllegalStateException si el ArrayList está vacío.
- buscarIndicePista(codigo: String): int	Valida el código y recorre el ArrayList por índices. Devuelve el índice encontrado o -1. Compara textos sin distinguir mayúsculas y minúsculas.

Las consultas de máximos devuelven referencias a objetos existentes, no copias de sus datos. En un empate se conserva el primero encontrado: el de menor índice en ubicaciones y el primero del orden actual del ArrayList para pistas.

getUbicaciones y getPistas devuelven copias superficiales de los contenedores. Agregar o quitar elementos de esas copias no altera las colecciones internas; los objetos contenidos siguen siendo las mismas instancias. Sus atributos private y sus métodos de actualización conservan las validaciones.

Tipos y métodos de Principal

Firma y visibilidad	Propósito y resultado
+ Principal()	Crea el Scanner y deja casoActual en null hasta que el usuario registre el primer caso.
+ main(args: String[]): void	Método static de entrada. Crea una instancia de Principal y llama a ejecutar. No necesita argumentos de consola para funcionar.
+ ejecutar(): void	Solicita el caso inicial y mantiene el menú hasta salir. Maneja errores recuperables y cierra el Scanner en un finally exterior.
- mostrarMenu(): void	Imprime las trece opciones numeradas de la guía.
- ejecutarOpcion(opcion: int): void	Solicita los datos de cada opción y llama a Caso. La opción 13 termina el ciclo de ejecutar; las demás realizan la operación indicada.
- solicitarCaso(): Caso	Lee nombre, código y detective; construye y devuelve un caso válido. Reintenta si los datos del caso son inválidos.
- leerEntero(mensaje: String): int	Muestra el mensaje y usa nextInt. Maneja InputMismatchException, limpia la línea incorrecta y repite la lectura.
- leerTexto(mensaje: String): String	Lee con nextLine y quita espacios de los extremos. Vuelve a solicitar el dato si queda vacío.
- mostrarReporte(): void	Muestra los siete resultados solicitados. Si faltan elementos, informa que no hay datos para máximos o promedio.

Distribucion de responsabilidades

Principal mostrará mensajes y solicitará datos. Las clases Caso, Ubicacion y Pista no crearán lectores Scanner ni pedirán información por consola. Sus excepciones se comunicarán a Principal para que este explique el problema y permita continuar.

Los métodos de consulta que devuelven objetos podrán devolver null cuando no exista un resultado. Los métodos boolean permitirán distinguir entre una modificación o eliminación realizada y una pista o ubicación inexistente, según sus contratos. int se usará para posiciones, cantidades y niveles; double se usará para el promedio.

3 Arreglos básico de ubicaciones

La propiedad ubicaciones de Caso será de tipo Ubicacion[] y se inicializará con new Ubicacion[MAX_UBICACIONES], donde MAX_UBICACIONES vale 5. Al crearlo, Java deja automáticamente las cinco referencias en null; no crea cinco objetos Ubicacion.

Su tamaño permanecerá fijo. Una posición contendrá una referencia a una ubicación registrada o null si está disponible. La cantidad de ubicaciones existentes se calculará contando las posiciones no nulas, no utilizando únicamente length, que siempre será 5.

4 Colección dinámica de pistas

La propiedad pistas de Caso será de tipo ArrayList<Pista> y se inicializará con new ArrayList<Pista>(). Comenzará vacía y crecerá o disminuirá con add y remove. No tendrá un máximo fijo definido por el enunciado; estará limitada por la memoria disponible.

Solo almacenará objetos Pista válidos y no nulos. Se utilizará ArrayList también en los parámetros o retornos que requieran esta colección. No se utilizará List.

5 Modificaciones de visibilidad

Las cuatro clases serán public y cada una tendrá su archivo Java. Los atributos serán private para impedir asignaciones directas desde otras clases. Los constructores, consultas y operaciones necesarias para utilizar el modelo serán public.

validarPosicion y buscarIndicePista serán private porque solo apoyan operaciones internas de Caso. En Principal, los métodos auxiliares del menú y de lectura serán private. main será public static; ejecutar y el constructor serán public.

Serán final el Scanner, los metadatos y contenedores de Caso, el código, nombre y dirección de Ubicacion, y el código de Pista. No habrá miembros protected porque el diseño no utiliza herencia.

6 Parámetros de los métodos

Las firmas de la respuesta 2 especifican todos los parámetros. Los constructores reciben los datos iniciales de cada objeto. Las operaciones del arreglo reciben posición de tipo int; el registro recibe además una Ubicación. Las operaciones sobre una pista existente reciben su código de tipo String; registrarPista recibe un objeto Pista.

Las modificaciones reciben todos los valores que pueden cambiar. Los métodos de lectura de Principal reciben un mensaje String; ejecutarOpcion recibe la opción int. Las consultas generales, los conteos, los máximos, el promedio y toString no requieren parámetros porque utilizan los atributos del objeto. main recibe String[] args por la firma de entrada de Java.

7 Inicialización y validación de objetos

Los valores iniciales se proporcionarán mediante constructores con parámetros. Principal se construirá sin parámetros y creará el único Scanner. Después solicitará un Caso válido. Cada Caso nuevo tendrá cinco espacios null y un ArrayList vacío, aunque existiera información en el caso anterior.

La referencia casoActual se reemplazará después de que el constructor de Caso termine correctamente. Si los datos del nuevo caso son inválidos, el anterior seguirá activo. La aplicación trabajará en memoria: la guía no exige persistencia en archivos.

Dato o condición	Validación antes de almacenar o modificar
Textos obligatorios	No null y no vacíos después de trim.
Riesgo de ubicación	Entero de 1 a 10 inclusive.
Importancia de pista	Entero de 1 a 10 inclusive.
Confiabilidad de pista	Entero de 0 a 100 inclusive.
Posición del arreglo	Índice de 0 a 4; libre para registrar y ocupado para modificar.
Pista nueva	Objeto no null y código no repetido en el caso.
Ubicación nueva	Objeto no null y datos validados por su constructor.

Cada método actualizar verificará todos los nuevos valores antes de asignar cualquiera. Así, si la confiabilidad es inválida, por ejemplo, la descripción y la importancia de la pista tampoco cambiarán. No se requieren setters independientes que permitan dejar una actualización a medias.

8 Comprobación de una posición vacía

Primero se verificará que el índice pertenezca al arreglo. Después se evaluará `ubicaciones[posicion] == null`. Si la condición es verdadera, el espacio está vacío; si es falsa, se puede consultar el objeto y utilizar sus métodos.

Para consultar todas las ubicaciones se recorrerán los índices de 0 a `ubicaciones.length - 1` y solo se mostrarán los elementos distintos de null, junto con su índice. Esta comprobación evita llamar métodos sobre una referencia nula.

9 Búsqueda modificación y eliminación de pistas

buscarIndicePista recorrerá desde 0 hasta pistas.size() - 1 y comparará el código solicitado con getCodigo() usando equalsIgnoreCase, después de recortar la entrada. Devolverá el índice encontrado o -1. No se utilizará == para comparar el contenido de los códigos.

Para modificar se buscará primero el índice. Si existe, se obtendrá la pista con get y se invocará actualizar. Para eliminar se localizará el índice y, terminado el recorrido de búsqueda, se ejecutará remove(indice). Las pistas posteriores se desplazarán una posición y size disminuirá.

Si el código no existe, la consulta devolverá null y la modificación o eliminación devolverá false. Se informará al usuario sin acceder a un índice -1. No se eliminarán elementos directamente dentro de un for-each, para evitar modificar la colección durante su recorrido.

10 Excepciones y uso de try catch Y finally

Situación	Excepción y tratamiento
Texto o decimal donde se espera un entero, o número fuera del rango de int	InputMismatchException en leerEntero. El catch consume la línea inválida con nextLine, muestra el error y vuelve a solicitar el número.
Niveles fuera de rango, texto obligatorio inválido, código de pista repetido, referencia null o posición inválida	IllegalArgumentException en constructores u operaciones del modelo. Un catch durante el registro inicial o dentro del ciclo del menú comunica la causa sin cambiar los datos válidos.
Registrar en un espacio ocupado, modificar una ubicación inexistente o pedir un promedio sin pistas	IllegalStateException en Caso. El menú la maneja con un catch y sigue funcionando. El reporte consulta primero contarPistas para evitar pedir el promedio vacío.
Fin de la entrada estándar	NoSuchElementException puede llegar desde Scanner. Se maneja en ejecutar para finalizar ordenadamente cuando ya no hay datos que leer.

Ubicación de los bloques de manejo

leerEntero tendrá un ciclo con try-catch para nextInt. Después de una lectura correcta consumirá el resto de la línea con nextLine, de modo que la siguiente lectura de texto no reciba el salto de línea pendiente. Cada respuesta numérica se solicitará en una línea independiente.

solicitarCaso manejará IllegalArgumentException al construir el caso y repetirá la solicitud. Dentro del ciclo principal, un try-catch alrededor de cada operación manejará IllegalArgumentException e IllegalStateException, mostrará el mensaje y volverá al menú.

ejecutar envolverá la sesión completa en un try con un catch para el fin de entrada y un finally que cierre el único Scanner. El cierre ocurrirá al abandonar la sesión, tanto por la opción Salir como por una salida excepcional. No se cerrará el Scanner después de cada opción, porque cerrarlo también cierra System.in.

Errores que se previenen mediante validaciones

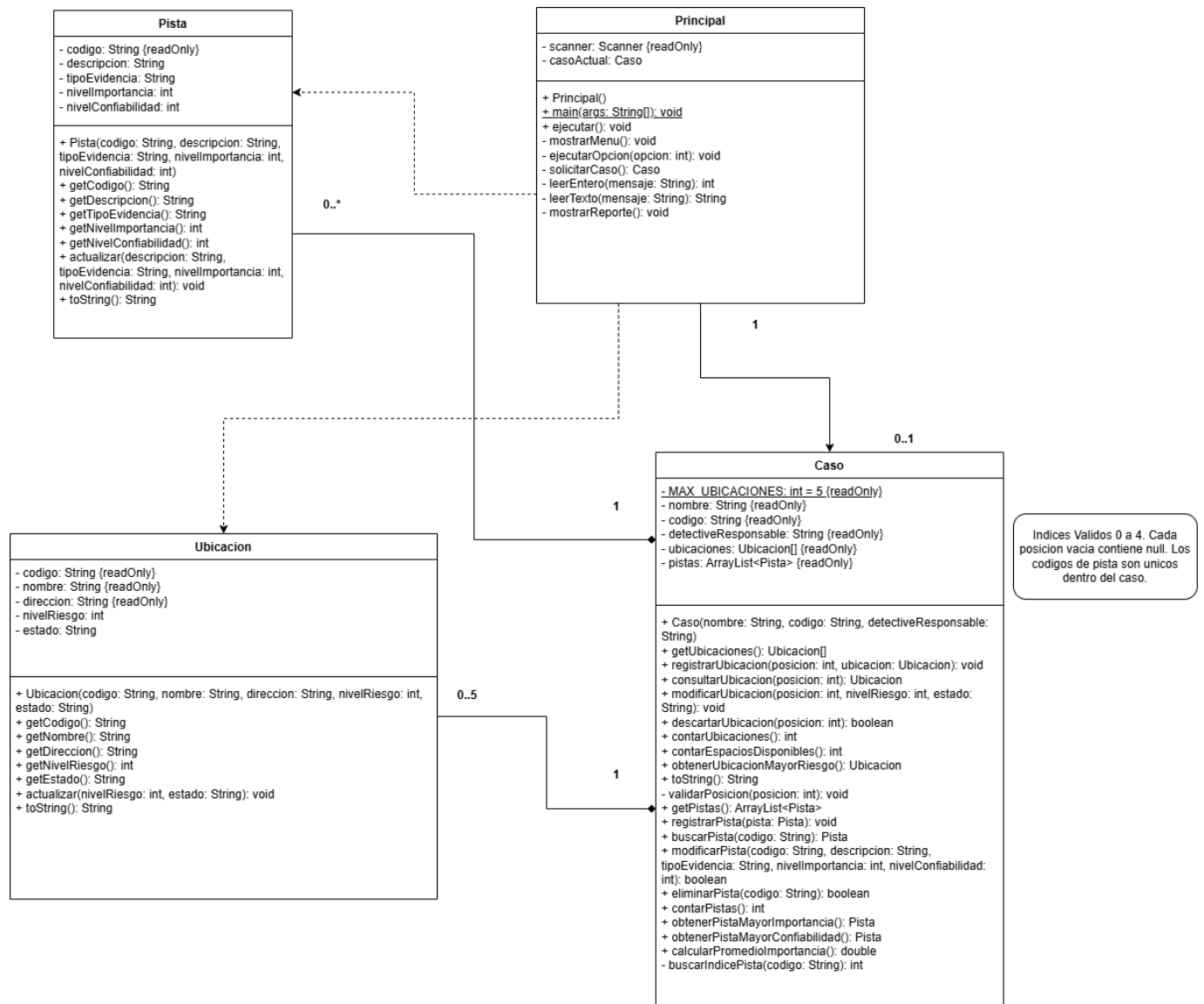
`ArrayIndexOutOfBoundsException` e `IndexOutOfBoundsException` se previenen comprobando los límites y el resultado de la búsqueda antes de acceder a arreglos o al `ArrayList`.

`NullPointerException` se previene comprobando `null`. `ConcurrentModificationException` se evita separando la búsqueda de la eliminación. No se usarán capturas generales de `Exception` para ocultar errores de programación.

El promedio no se calculará con una colección vacía. La validación se hará antes de dividir y no se dependerá de `ArithmeticException`, ya que una división de punto flotante entre cero no necesariamente la produce. Cuando no haya datos, el reporte mostrará esa condición; los conteos seguirán mostrando cero y los espacios disponibles serán cinco.

Diseño UML y relaciones entre clases

El diseño se presenta en una vista de relaciones y tres vistas de detalle del mismo modelo. Las páginas siguientes muestran todos los atributos y métodos definidos en el análisis. No se utiliza herencia.



Repositorio:

<https://github.com/Josue6864/laboratorio-2-agencia-detectives.git>